Backend · concept board

Python

Types, collections, functions, OOP, generators, async, the GIL, tooling

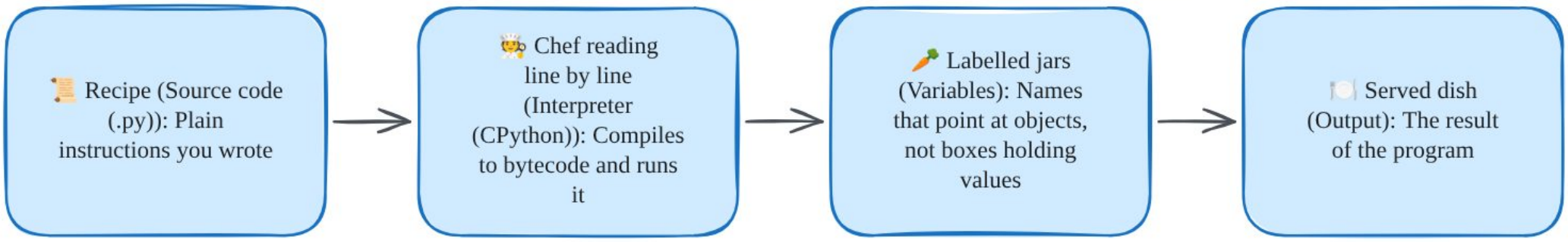
1. Python basics and data types
2. Functions, scope and decorators
3. Classes and OOP
4. Iterators, generators and context managers
5. Errors, files and modules
6. Concurrency, the GIL and async

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

1. Python basics and data types

Definition

Python is a dynamically typed, interpreted, indentation-based language. Everything is an object, and names are references to objects.



Think of Python like a kitchen with a recipe reader

Type	Example	Mutable?
int, float, bool	42, 3.14, True	no
str	"hello"	no
tuple	(1, 2)	no
list	[1, 2, 3]	yes
dict	{"a": 1}	yes (keys must be hashable)
set	{1, 2, 3}	yes (unique items)

Collections and comprehensions

```
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums if n % 2 == 1]      # [1, 9, 25]
by_len = {w: len(w) for w in ["a", "bb", "ccc"]}    # dict comprehension
unique = {n % 3 for n in nums}                      # set comprehension

first, *rest = nums                                 # unpacking
print(squares, by_len, unique, first, rest)

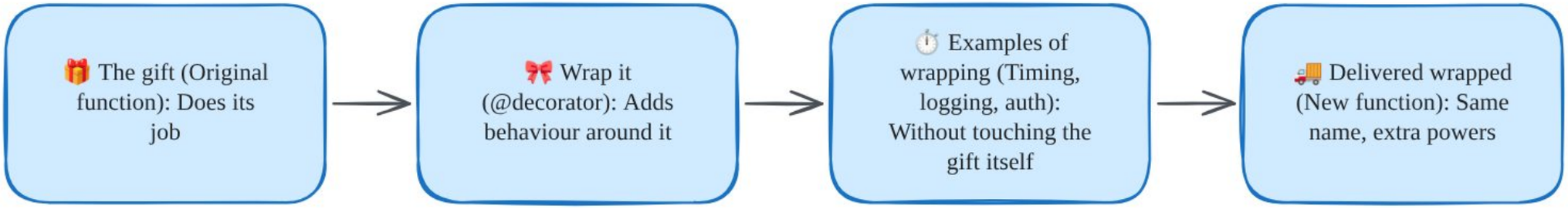
name = "Asha"
print(f"Hello {name!r}, {len(name)} letters")       # f-string
```

- The equality operator compares values, while is compares identity (same object). Use is None for None.
- Truthiness: empty [], {}, "", 0, None are falsy.
- Strings and tuples are immutable; lists, dicts and sets are mutable.
- Slicing: a[1:4], a[::-1] (reverse), a[-1] (last).

Mutable default argument

def add(x, items=[]) shares ONE list across all calls. Use items=None and create the list inside the function.

2. Functions, scope and decorators



Think of a decorator like gift wrapping

Arguments, closures and a decorator

```
import functools, time

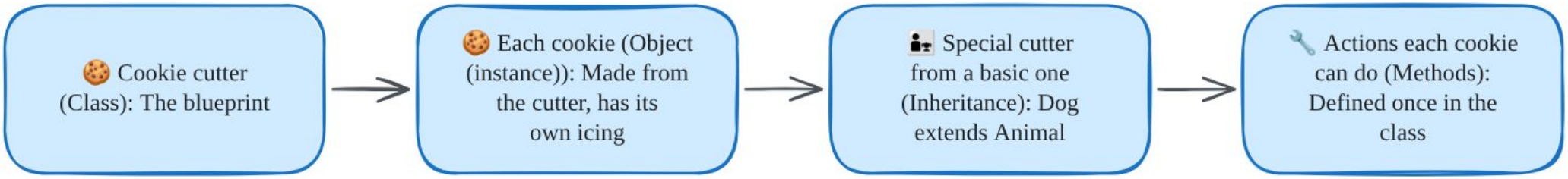
def greet(name, *args, greeting="Hi", **kwargs):
    return f"{greeting} {name} {args} {kwargs}"

def timed(fn):
    @functools.wraps(fn)                # keep the original name and docstring
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        print(f"{fn.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timed
# … 5 more lines in the notes
```

- *args collects extra positional arguments into a tuple, and the double-star parameter (kwargs) collects keyword arguments into a dict.
- LEGB scope order: Local → Enclosing → Global → Built-in. Use nonlocal / global to rebind.
- A decorator is a function that takes a function and returns a new one: @timed means slow_sum = timed(slow_sum).
- Functions are first-class objects: pass them, return them, store them in dicts.
- lambda x: x * 2 is a one-expression anonymous function.

3. Classes and OOP



Think of classes like cookie cutters

Classes, dataclasses and inheritance

```
from dataclasses import dataclass

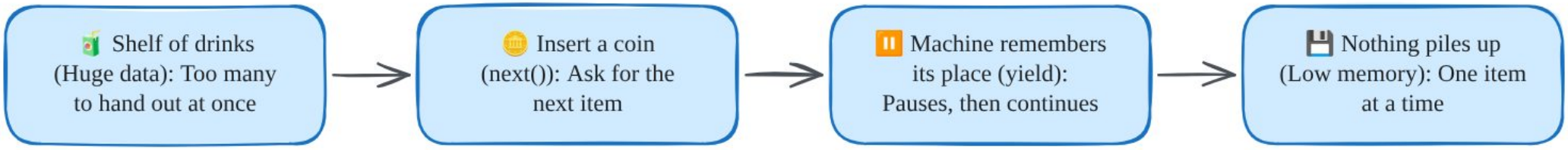
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        raise NotImplementedError
    def __repr__(self):
        return f"{type(self).__name__}({self.name!r})"

class Dog(Animal):
    def speak(self):
        return "Woof"

@dataclass
# … 5 more lines in the notes
```

- self is the instance; @classmethod gets the class (cls); @staticmethod gets neither.
- Dunder methods customise behaviour: __str__, __eq__, __len__, __iter__, __enter__/__exit__.
- @property exposes a method as an attribute (computed or validated).
- Multiple inheritance resolves by the MRO (method resolution order, Class.__mro__).
- Prefer composition over deep inheritance; use dataclass for plain data holders.

4. Iterators, generators and context managers



Think of a generator like a vending machine

Generators are lazy

```
def fib():
    a, b = 0, 1
    while True:
        yield a             # pauses here and remembers its state
        a, b = b, a + b

def take(n, it):
    for _, v in zip(range(n), it):
        yield v

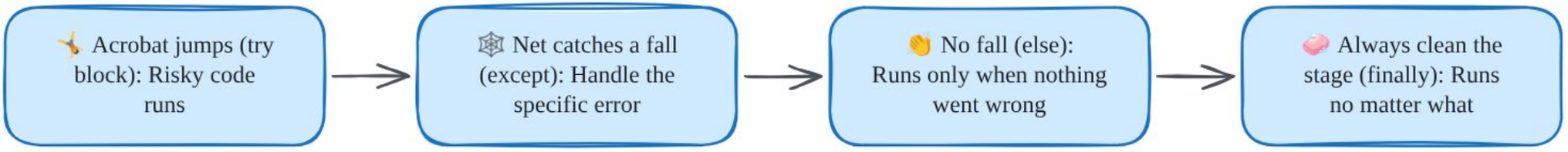
print(list(take(10, fib())))

squares = (n * n for n in range(10 ** 9))   # generator expression: no memory used
print(next(squares), next(squares))

# … 12 more lines in the notes
```

- An iterable has __iter__; an iterator also has __next__ and is consumed once.
- yield turns a function into a generator: values are produced on demand, so huge or infinite streams use almost no memory.
- A context manager (with) guarantees cleanup (files, locks, DB connections) even when errors happen.
- Useful tools: enumerate, zip, itertools, collections.Counter, defaultdict, deque.

5. Errors, files and modules



Think of try/except like a safety net at a circus

try / except / else / finally

```
class NotEnoughStock(Exception):
    pass

def buy(stock, qty):
    if qty > stock:
        raise NotEnoughStock(f"only {stock} left")
    return stock - qty

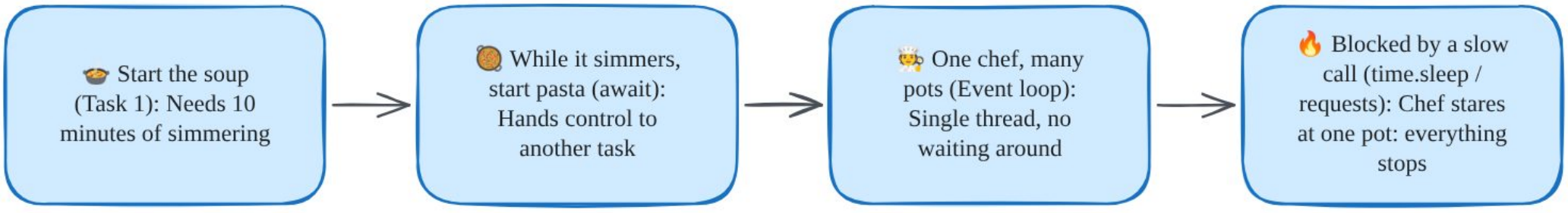
try:
    left = buy(3, 5)
except NotEnoughStock as e:
    print("failed:", e)
else:
    print("ok", left)              # runs only when no exception
finally:
    print("always runs")
```

- Catch specific exceptions; a bare except: hides bugs. Re-raise with raise and chain with raise X from e.
- Read files with with open(path, encoding="utf-8") as f: and iterate lines lazily.
- A module is a .py file, a package is a folder with modules. The __main__ guard at the bottom of a file runs code only when the file is executed directly.
- Virtual environments (python -m venv .venv) isolate dependencies; list them in requirements.txt or pyproject.toml.
- Use type hints (def f(x: int) -> str) with mypy or pyright for safer code; they are not enforced at runtime.

6. Concurrency, the GIL and async

The GIL

CPython has a Global Interpreter Lock: only one thread runs Python bytecode at a time. Threads help with I/O-bound work, but CPU-bound work needs multiprocessing.



Think of asyncio like a chef cooking several dishes

Tool	Best for	Notes
threading	I/O-bound (network, files)	limited by the GIL for CPU work
multiprocessing	CPU-bound	separate processes, real parallelism
asyncio	many concurrent I/O tasks	single thread, cooperative await

asyncio runs waits concurrently

```
import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)        # yields control while "waiting"
    return f"{name} done"

async def main():
    results = await asyncio.gather(fetch("a", 1), fetch("b", 1), fetch("c", 1))
    print(results)                    # all three finish in about 1 second

asyncio.run(main())
```

Common mistake

Calling a blocking function (time.sleep, requests.get, a slow DB driver) inside async code freezes the whole event loop. Use async libraries or run_in_executor.

Revise

Mutable vs immutable, is vs equality, no mutable defaults, star-args and kwargs, decorators, generators for lazy streams, with for cleanup, GIL → threads for I/O and processes for CPU, asyncio for many waits, venv + type hints.

Rapid-fire Q&A (1/2)

List vs tuple?

Lists are mutable and for homogeneous, changing collections; tuples are immutable, hashable (if their items are) and used for fixed records and dict keys.

What does is do versus the equality operator?

Equality compares values; is checks that two names refer to the same object. Use is for None.

What is the mutable default argument trap?

Default values are evaluated once at definition time, so a default list or dict is shared between calls. Use None and create it inside.

What are args and kwargs in a function signature?

A single-star parameter gathers extra positional arguments into a tuple; a double-star parameter gathers extra keyword arguments into a dict.

What is a decorator?

A function that wraps another function to add behaviour; @d above def f is the same as f = d(f). Use functools.wraps to keep metadata.

Generator vs list?

A generator yields items lazily one at a time (low memory, single pass); a list builds everything in memory at once.

Rapid-fire Q&A (2/2)

What is the GIL?

A lock in CPython that lets only one thread execute Python bytecode at a time. It limits CPU-bound threading, so use multiprocessing for CPU work.

When use asyncio, threads or processes?

asyncio for many I/O waits with async libraries, threads for blocking I/O, processes for CPU-bound work.

What is a context manager?

An object used with with that sets up and always tears down a resource via __enter__/__exit__ (or @contextmanager).

Shallow vs deep copy?

A shallow copy (copy.copy) copies the container but shares nested objects; a deep copy (copy.deepcopy) copies everything recursively.

What are dunder methods?

Special methods like __init__, __repr__, __eq__ and __iter__ that let your objects work with built-in syntax and functions.

How does Python manage memory?

Reference counting frees objects when the count hits zero, plus a cyclic garbage collector for reference cycles.